



5

FLOCKING BOIDS



Look closely at a flock of birds or a school of fish, and you'll notice that although the group is composed of individual creatures, the group as a whole seems to have a life of its own. The birds in a flock align with each other as they move and flow over and around obstacles. They break formation when disturbed or startled, but then they regroup, as if controlled by some larger force.

In 1986, Craig Reynolds created a realistic-looking simulation of the flocking behavior of birds called the *Boids model*. One remarkable thing about the Boids model (named after the stereotypical New Yorker's pronunciation of the word *birds*) is that only three simple rules govern the interaction between individuals in the flock, yet the model produces remarkably realistic flocking behavior. The Boids model is widely studied and has even been used to animate computer-generated swarms like the marching penguins in the movie *Batman Returns* (1992).

In this project, you'll use Reynolds's three rules to create a Boids simulation of the flocking behavior of N birds and plot their positions and

directions of movement over time. You'll also provide a method to add a bird to the flock, as well as a scatter effect that you can use to study the impact of a local disturbance on the flock. Boids is called an N -body simulation because it models a dynamic system of N particles that exert forces on each other.

How It Works

The three core rules of the Boids simulation are as follows:

Separation Keep a minimum distance between the boids.

Alignment Point each boid in the average direction of movement of its local flockmates.

Cohesion Move each boid toward the center of mass of its local flockmates.

Boids simulations can add other rules too, such as ones to avoid obstacles or scatter the flock when it's disturbed, as you'll learn in the following sections. To create the Boids animation, you'll do the following for every time step in the simulation:

1. 1. For all boids in the flock:
 1. Apply the three core rules.
 2. Apply any additional rules.
 3. Apply all boundary conditions.
2. Update the positions and velocities of the boids.
3. Plot the new positions and velocities.

As you'll see, these simple steps create a flock with evolving, complex behavior.

Requirements

These are the Python modules you'll be using in this simulation:

- `numpy` arrays to store the positions and velocities of the boids
- The `matplotlib` library to animate the boids
- `argparse` to process command line options
- The `scipy.spatial.distance` module, which has some really neat methods for calculating distances between points

I chose to use `matplotlib` for boids as a matter of simplicity and convenience. To draw a huge number of boids as quickly as possible, you might use something like the OpenGL library. We'll explore graphics in more detail in [Part III](#) of this book.

The Code

You'll encapsulate the behavior of a group of boids in a class called `Boids`. First you'll set the initial positions and velocities of the boids. Next, you'll set up the boundary conditions for the simulation, look at how the boids are drawn, and implement the Boids simulation rules discussed earlier. Finally, you'll add some interesting events to the simulation by allowing the user to add boids and scatter the flock. To see the full project code, skip ahead to [“The Complete Code”](#) on [page 96](#). You can also download it from the book's GitHub repository at <https://github.com/mkvenkit/pp2e/blob/main/boids/boids.py>.

Initializing the Simulation

The Boids simulation needs to compute the position and velocities of the boids at each step by pulling information from `numpy` arrays. At the beginning of the simulation, you use the `Boids` class's `__init__()` method to create those arrays and initialize all boids in approximately the center of the screen, with their velocities set in random directions.

```
import argparse
```

```
import math

import numpy as np

import matplotlib.pyplot as plt

import matplotlib.animation as animation

from scipy.spatial.distance import squareform, pdist

from numpy.linalg import norm

❶ width, height = 640, 480

class Boids:

    """class that represents Boids simulation"""

    def __init__(self, N):

        """initialize the Boids simulation"""

        # init position & velocities

        ❷ self.pos = [width/2.0, height/2.0] +

            10*np.random.rand(2*N).reshape(N, 2)

        # normalized random velocities

        ❸ angles = 2*math.pi*np.random.rand(N)

        ❹ self.vel = np.array(list(zip(np.cos(angles), np.sin(angles))))

        self.N = N
```

First you import the modules required for the program and set the width and height of the simulation window on the screen ❶. Then you begin declaring the `Boids` class. Within the class's `__init__()` method, you create a `numpy` array called `pos` to store the x- and y-coordinates of all the boids ❷. For the initial value of each pair of coordinates, you start with the center of the window, `[width/2.0, height/2.0]`, and add a random displacement of up to 10 units. The code `np.random.rand(2*N)` creates a one-dimensional array of $2N$ random numbers in the range $[0, 1]$, which you multiply by 10 to scale to a range of $[0, 10]$. The `reshape()` call converts the one-dimensional array into a two-dimensional array of shape $(N, 2)$, perfect for storing N pairs of x- and y-coordinates. Notice, too, the `numpy` broadcasting rules in action here: the 1×2 array `[width/2.0, height/2.0]` representing the center of the window is added to each element in the $N \times 2$ array to randomly offset each boid's position from the center.

Next, you create an array of random unit velocity vectors (these are vectors of magnitude 1.0, pointing in random directions) for each boid using the following method: given an angle t , the pair of numbers $(\cos(t), \sin(t))$ lie on a circle of radius 1.0, centered at the origin $(0, 0)$. If you draw a line from the origin to a point on this circle, it becomes a unit vector that depends on the angle t . So if you choose t at random, you end up with a random velocity vector. **Figure 5-1** illustrates this scheme.

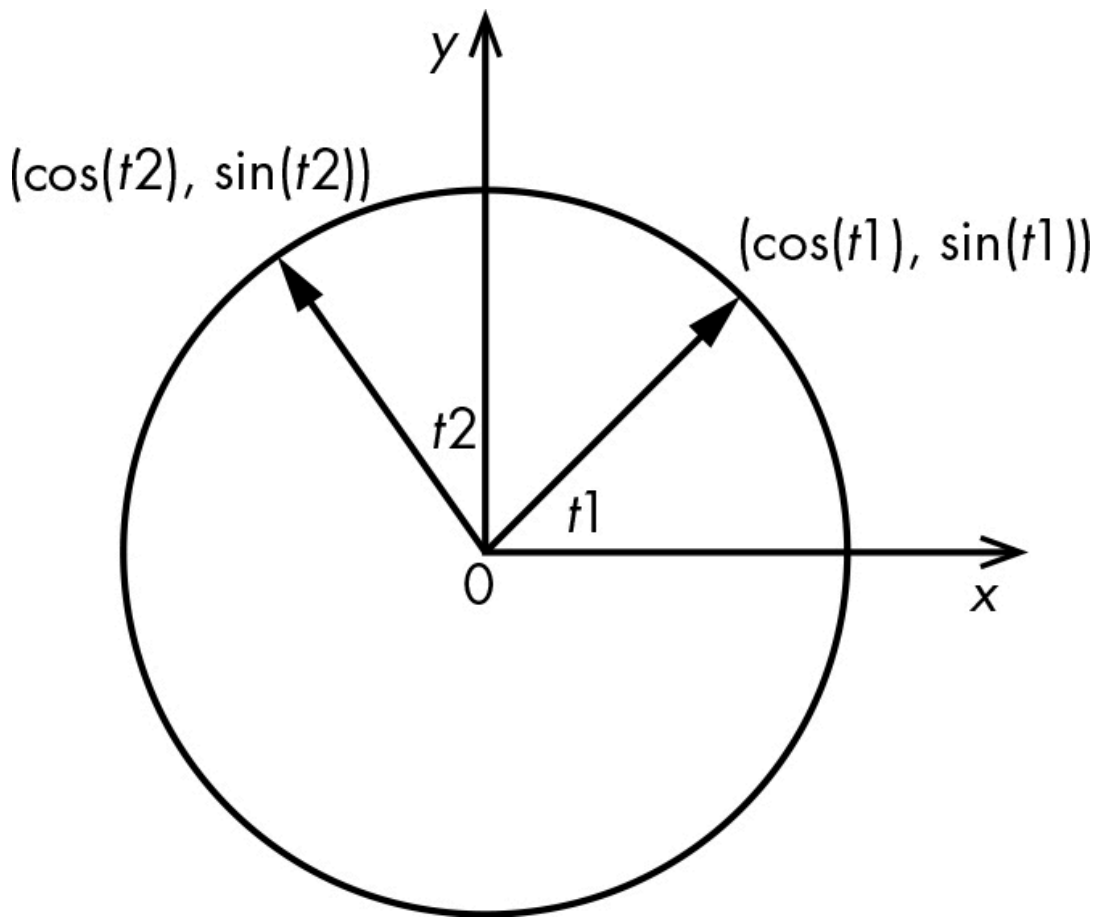


Figure 5-1: Generating random unit velocity vectors

Returning to the code, you put this method into action by first generating an array of N random angles in the range $[0, 2\pi]$ ③. Then you create an array of random unit velocity vectors by calculating the cosine and sine of those angles ④. You group the coordinates of each vector using Python's built-in `zip()` method. The following is a simple example of `zip()`. This joins two lists into a list of tuples. The `list()` is needed because just calling `zip` will create only an iterator—you need all the elements in the list.

```
>>> list(zip([0, 1, 2], [3, 4, 5]))
```

```
[(0, 3), (1, 4), (2, 5)]
```

In summary, you've generated two arrays that will be useful throughout the simulation, `pos` and `vel`. The first contains random positions

clustered within a 10-pixel radius around the center of the screen, and the second contains unit velocities pointing in random directions. This means that at the start of the simulation, the boids will all hover around the center of the screen, pointed in random directions.

The `__init__()` method continues by declaring some constant values that will help govern the simulation:

```
# min dist of approach
```

```
❶ self.minDist = 25.0
```

```
# max magnitude of velocities calculated by "rules"
```

```
❷ self.maxRuleVel = 0.03
```

```
# max magnitude of final velocity
```

```
❸ self.maxVel = 2.0
```

Here you define the minimum distance of approach between two boids ❶. You'll use this value later to apply the separation rule. Then you define `maxRuleVel`, which limits how much a boid's velocity can be changed each time one of the simulation rules is applied ❷. You also define `maxVel`, which sets an overall limit on the boids' velocities ❸.

Setting Boundary Conditions

Birds fly in a boundless sky, but the boids must play in limited space. To create that space, you'll set boundary conditions, as you did with the toroidal boundary condition in the Game of Life simulation in [Chapter 3](#). In this case, you'll apply a *tilled boundary condition* (actually the continuous space version of the boundary condition you used in [Chapter 3](#)).

Think of the Boids simulation as taking place in a tiled space: when a boid moves out of a tile, it moves in from the opposite direction to an identical tile. The main difference between the toroidal and tiled boundary conditions is that this Boids simulation won't take place on a discrete grid; instead, the birds move over a continuous region. [Figure 5-2](#) shows what those tiled boundary conditions look like.

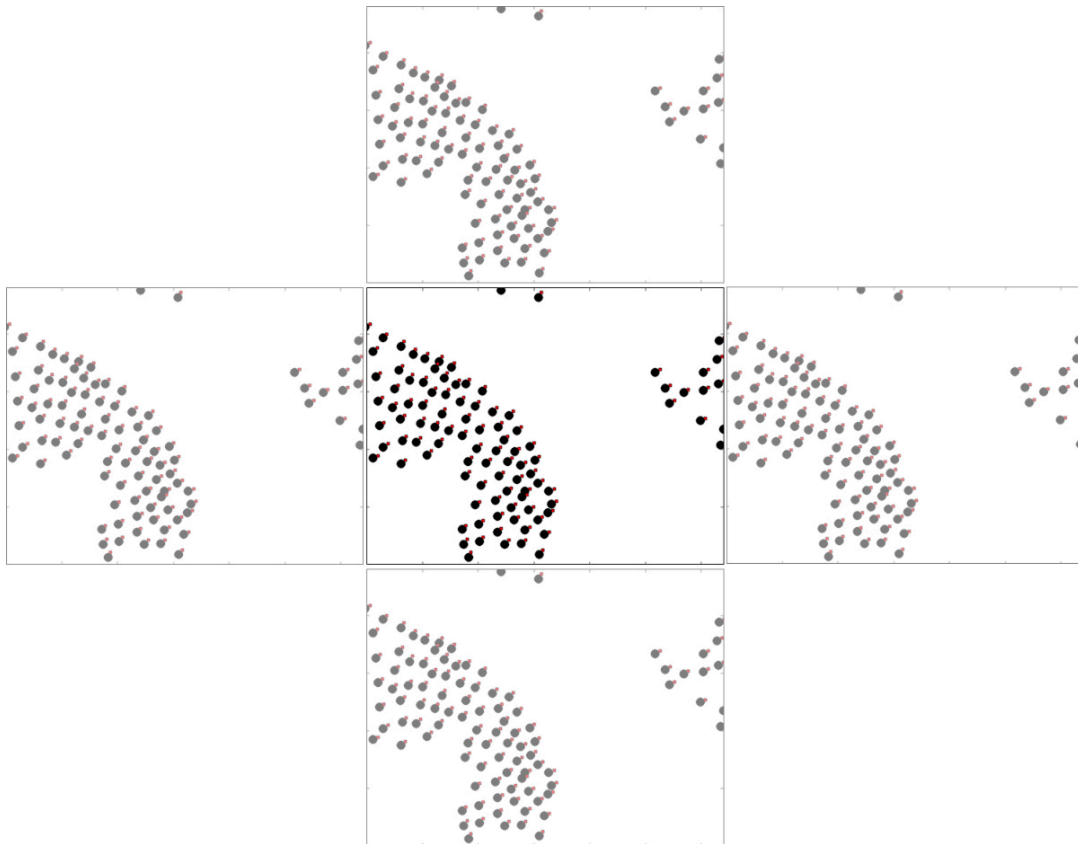


Figure 5-2: *Tiled boundary conditions*

Look at the tile in the middle. The birds flying out to the right are entering the tile on the right, but the boundary conditions ensure that they actually come right back into the center tile through the tile at the left. You can see the same thing happening at the top and bottom tiles.

You implement the tiled boundary conditions for the Boids simulation as a method on the `Boids` class:

```
def applyBC(self):
```



```
"""apply boundary conditions"""
```

```
deltaR = 2.0
```

```
for coord in self.pos:
```

```
    ❶ if coord[0] > width + deltaR:
```

```
        coord[0] = - deltaR
```

```
    if coord[0] < - deltaR:
```

```
        coord[0] = width + deltaR
```

```
    if coord[1] > height + deltaR:
```

```
        coord[1] = - deltaR
```

```
    if coord[1] < - deltaR:
```

```
        coord[1] = height + deltaR
```

This method applies the tiled boundary conditions to each set of boid coordinates in the `pos` array. For example, if an x-coordinate is greater than the width of the window ❶, you set it back to the left edge of the window. The `deltaR` in this line provides a slight buffer, which allows the boid to move slightly outside the window before it starts coming back in from the opposite direction, thus producing a better visual effect. You perform a similar check at the left, top, and bottom edges of the window.

Drawing a Boid

To build the animation, you need to know each boid's position and velocity and have a way to indicate both the position and direction of motion at each time step.

Plotting the Boid's Body and Head

To animate the boids, you use `matplotlib` and a little trick to plot both the position and velocity. Draw each boid as two circles, as shown in **Figure 5-3**. The larger circle represents the body, and the smaller one represents the head. Point P marks the center of the body. For our purposes, you can consider P to be the position of the boid, and you'll set it using coordinates from the `pos` array discussed earlier. Point H is the center of the head. You calculate the position of H according to the formula $H = P + k \times V$, where V is the velocity of the boid and k is a constant representing the distance from the center of the body to the center of the head. This way, the boid's head will be aligned with its direction of motion at any given time, which visually communicates the boid's direction of movement more clearly than just drawing the body alone.

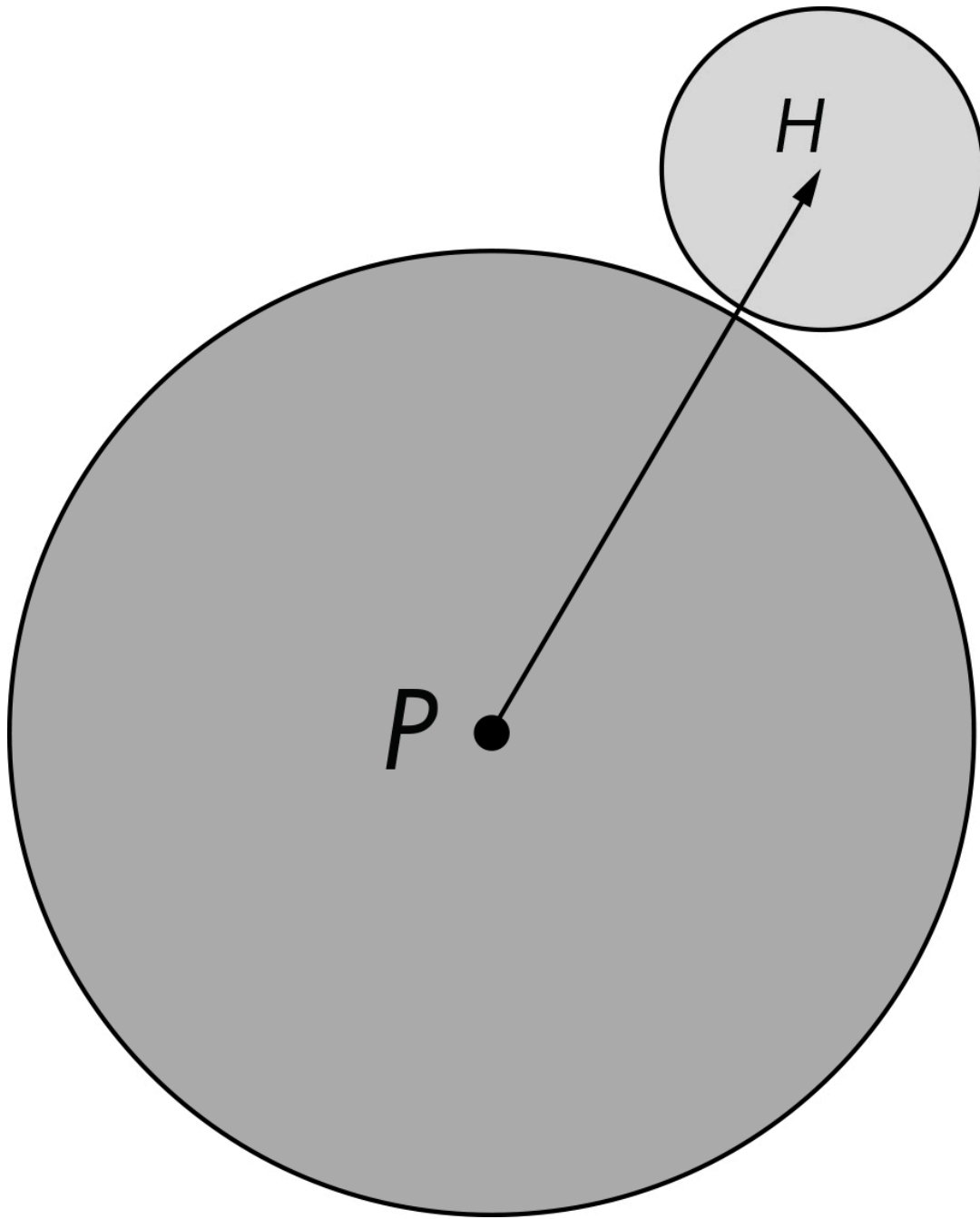


Figure 5-3: Representing a boid

In the following snippet from the program's `main()` function, you draw the boid's body and head as circular markers using `matplotlib`:

```
fig = plt.figure()
```

```
ax = plt.axes(xlim=(0, width), ylim=(0, height))
```

```
① pts, = ax.plot([], [], markersize=10, c='k', marker='o', ls='None')
```

```

❷ head, = ax.plot([], [], markersize=4, c='r', marker='o', ls=None')

❸ anim = animation.FuncAnimation(fig, tick, fargs=(pts[0], head,
boids),

                                interval=50)

```

You set the size and shape of the markers for the boid's body (`pts`) ❶ and head (`head`) ❷. The `'k'` and `'r'` strings specify the colors black and red, respectively, and `'o'` produces a circular marker. The `ax.plot()` method returns a list of `matplotlib.lines.Line2D` objects. The `,` syntax in these lines picks up the first and only element in this list.

You next initialize a `matplotlib.animation.FuncAnimation()` object ❸, which sets up a callback function `tick()` to be called for every frame of the animation (we'll look at this function later in the chapter). The `fargs` parameter lets you specify the arguments of the callback function, and you also set the time interval (50 milliseconds in this case) at which this function will be called. Now that you know how to draw the body and the head, let's see how to update their positions.

Updating the Boid's Position

Once the animation starts, you need to update both the boid's position and the location of the head, which tells you the direction in which the boid is moving. You do so with this code:

```

vec = self.pos + 10*self.vel/self.maxVel

head.set_data(vec.reshape(2*self.N)[::2], vec.reshape(2*self.N)[1::2])

```

First you calculate the position of the head by applying the $H = P + k \times V$ formula discussed earlier. You use a k value of 10 units in the direction of the velocity (`vel`). You then update (`reshape`) the `mat-`

`plotlib` `axis ( set_data )` with the new values of the head position. The `[:2]` picks out the even-numbered elements (x-axis values) from the velocity list, and the `[1:2]` picks out the odd-numbered elements (y-axis values).

Applying the Rules of the Boids

In this section, we'll look at how to implement the three rules of the Boids simulation—separation, alignment, and cohesion—to recalculate the boids' velocities at each time step. We'll start by focusing just on the separation rule. The goal is to generate a new velocity vector for each boid that pushes it away from its nearby flockmates, defined as all the boids within a certain radius R . Given two boids i and j with positions P_i and P_j , $P_i - P_j$ produces a new velocity vector for boid i that points away from boid j . We'll call this a *displacement vector*. To calculate a new velocity vector V_i for boid i that on average pushes it away from *all* its nearby flockmates, simply sum all the displacement vectors between boid i and each boid within radius R . In other words, $V_i = (P_i - P_1) + (P_i - P_2) + \dots (P_i - P_N)$, provided the distance between boids i and j is less than R . You can write this more formally as:

$$V_i = \begin{cases} \sum (P_i - P_j) & \text{if } |P_i - P_j| < R \\ 0 & \text{otherwise} \end{cases}$$

Notice that implementing this rule—and, indeed, implementing the other Boids rules as well—involves calculating the distance between each boid and every other boid to determine which boids are local flockmates. The traditional way to do this in Python would be to use a pair of nested loops to iterate through the boids. As you'll see, however, `numpy` arrays provide more efficient methods that bypass the need for loops. We'll implement both approaches and compare the results and then apply what we've learned to the actual simulation code.

Using Nested Loops

First, let's define a function `test1()` that implements the separation rule in a straightforward way, using loops:

```
def test1(pos, radius):

    # fill output with zeros

    vel = np.zeros(2*N).reshape(N, 2)

    # for each pos
    ❶ for (i1, p1) in enumerate(pos):

        # velocity contribution

        val = np.array([0.0, 0.0])

        # for each other pos
        ❷ for (i2, p2) in enumerate(pos):

            if i1 != i2:

                # calculate distance from p1

                dist = math.sqrt((p2[0]-p1[0])*(p2[0]-p1[0]) +

                                (p2[1]-p1[1])*(p2[1]-p1[1]))

                # apply threshold

                ❸ if dist < radius:

                    ❹ val += (p2 - p1)

    # set velocity
```

```

    vel[i1] = val

# return computed velocity

return vel

```

This code uses a nested pair of loops. The outer loop ❶ goes through each boid in the `pos` array. The inner loop ❷ computes the distance between the current boid and each other boid in the array. If the distance is less than the threshold defined as the function's `radius` parameter ❸, you calculate the displacement vector as discussed earlier and add the result to `val` ❹. At the end of each cycle of the inner loop, `val` holds a new velocity that will push the current boid away from its neighbors. You store that velocity back in the `vel` array.

Using numpy Methods

Now let's define a function `test2()` that does the same thing “the numpy way,” avoiding loops and using highly optimized numpy methods. You'll also use methods from the `scipy.spatial.distance` module to efficiently calculate the distance between points:

```

def test2(pos, radius):

    # get distance matrix

    ❶ distMatrix = squareform(pdist(pos))

    # apply threshold

    ❷ D = distMatrix < radius

    # compute velocity

    ❸ vel = pos*D.sum(axis=1).reshape(N, 1) - D.dot(pos)

```

```
return vel
```

You use the `squareform()` and `pdist()` methods (defined in the `scipy` library) to calculate the distances between every possible pair of points in the `pos` array ❶. For an array of N points, `squareform()` gives you an $N \times N$ matrix, where any given entry M_{ij} is the distance between points P_i and P_j . Let's consider a quick example of how that looks. In this code, you call the methods on an array of three points:

```
>>> import numpy as np

>>> from scipy.spatial.distance import squareform, pdist

>>> x = np.array([[0.0, 0.0], [1.0, 1.0], [2.0, 2.0]])

>>> squareform(pdist(x))

array([[0.      , 1.41421356, 2.82842712],
       [1.41421356, 0.      , 1.41421356],
       [2.82842712, 1.41421356, 0.      ]])
```

Since you provide an array of three points, the result is a 3×3 matrix of distance calculations. The values in the first row, for example, tell you the distance between the first point (`[0.0, 0.0]`) and each point in the array. The zeros running diagonally down the array correspond to the distance between each point and itself.

Returning to the `test2()` function, you next filter the matrix based on whether the distance is less than the specified `radius` ❷. Using the same example array of three points, you have the following:

```
>>> squareform( pdist(x)) < 1.4

array([[ True, False, False],
```


[False, True, False],

[False, False, True]])

The `<` comparison creates a Boolean matrix of `True/False` values corresponding to the original distance matrix—`True` if a distance is less than the given threshold (in this example, 1.4).

Back in `test2()`, you use a modified version of the equation for V_i discussed earlier, broadcast over the entire `pos` array ❸. That equation can be rewritten as:

$$V_i = mP_i - \sum_{d < D} P_k$$

Here, the second summation term on the right includes only the points P that satisfy the distance criteria. The number of elements in the summation term is m . This equation can again be written as:

$$V_i = mP_i - \sum D_{ij} P_j$$

where D_{ij} is row i of the Boolean matrix you generated ❷, m is the number of `True` values in that row, and P_j is all the points P that fall within the specified radius of the current boid.

The `D.sum` method ❸ adds up the `True` values in the Boolean matrix in a column-wise fashion, giving you m from the equation. The reshape is required because the result of the sum is a one-dimensional array of N values (shape $(N,)$), and you want it to be of shape $(N, 1)$ so it's compatible for multiplication with the position array. The `D.dot(pos)` part of the line ❸ is taking the dot product (multiplication) of the Boolean matrix and the array of boid positions, which corresponds to the $D_{ij}P_j$ part of the equation.

Comparing Approaches

Comparing the two approaches, `test2()` is much more compact than `test1()`, but its real advantage is speed. Let's use the Python `timeit` module to evaluate the performance of the two functions. First, enter the code for the functions `test1()` and `test2()` in a file named *test.py*, as shown here:

```
import math

import numpy as np

from scipy.spatial.distance import squareform, pdist, cdist

N = 100

width, height = 640, 480

pos = np.array(list(zip(width*np.random.rand(N),
height*np.random.rand(N))))

def test1(pos, radius):

    -- snip --

def test2(post, radius):

    -- snip --
```

Now use the `timeit` module in a Python interpreter session to compare the performance of the two functions:

```
>>> from timeit import timeit

>>> timeit('test1(pos, 100)', 'from test import test1, N,
pos, width, height', number=100)
```

7.880876064300537

```
>>> timeit('test2(pos, 100)', 'from test import test2, N,  
pos, width, height', number=100)
```

0.036969900131225586

On my computer, the `numpy` code without loops runs about 200 times faster than the code that uses explicit loops! But why? Aren't they both doing more or less the same thing?

As an interpreted language, Python is inherently slower than compiled languages like C. The `numpy` library brings the convenience of Python and performance nearly equal to that of C by providing highly optimized methods that operate on arrays of data. In general, you'll find that `numpy` works best when you reorganize your algorithm as steps that operate on entire arrays at once, without looping through individual elements to perform computations.

Writing the Final Method

Now that you've compared the two approaches, you're ready to use what you've learned to write a final version of the method that applies all three rules of the simulation and returns updated velocities for all the boids. The `applyRules()` method, part of the `Boids` class, uses the optimized `numpy` techniques discussed earlier.

```
def applyRules(self):
```

```
    # get pairwise distances
```

```
    ❶ self.distMatrix = squareform(pdist(self.pos))
```

```
    # apply rule #1: separation
```

```
    D = self.distMatrix < self.minDist
```

```
❷ vel = self.pos*D.sum(axis=1).reshape(self.N, 1) - D.dot(self.pos)

❸ self.limit(vel, self.maxRuleVel)

# distance threshold for alignment (different from separation)

❹ D = self.distMatrix < 50.0

# apply rule #2: alignment

❺ vel2 = D.dot(self.vel)

self.limit(vel2, self.maxRuleVel)

❻ vel += vel2

# apply rule #3: cohesion

❼ vel3 = D.dot(self.pos) - self.pos

self.limit(vel3, self.maxRuleVel)

❽ vel += vel3

return vel
```

You compute the pairwise distance matrix between the boids using the `squareform()` and `pdist()` methods from the `scipy` library, as discussed earlier ❶. When you apply the separation rule using `numpy` methods ❷, each boid is pushed away from neighboring boids within a distance of `minDist` (25 pixels). The calculated velocities are clamped, or restricted, to a certain maximum value using the `Boids` class's `limit()` method ❸, which we'll look at later. Without this restriction, the velocities would increase with each time step, and the simulation would go haywire.

You next generate a new Boolean matrix using a distance threshold of 50 pixels rather than 25 ④. You'll use this broader definition of neighboring flockmates to apply the alignment and cohesion rules. The alignment rule is implemented so that each boid is influenced by and aligns itself with the average velocity of its neighbors. You get that average simply by taking the dot product of `D` (the Boolean matrix) and the velocity array ⑤. Once again, you restrict the calculated velocities to a maximum so they don't increase indefinitely. (Using the compact `numpy` syntax makes all these computations simple and fast.)

Finally, you apply the cohesion rule by adding the positions of all the neighboring boids and then subtracting the position of the current boid ⑦. This produces a velocity vector for each boid that points to the *centroid* or geometric center of its neighbors. Again, you limit the velocities to keep them from getting out of control.

Each of the three rules produces its own velocity vector for each boid. At ⑥ and ⑧, you add these vectors together, producing an overall velocity vector for each boid that reflects the influence of all three simulation rules. You store the final velocity vectors in the `vel` array.

Limiting the Velocities

In the previous section, you saw how the `limit()` method was called after applying each rule to keep the boids' velocities from getting out of control. Here's that method:

```
def limit(self, X, maxVal):

    """limit the magnitude of 2D vectors in array X to maxVal"""

    ❶ for vec in X:

        self.limitVec(vec, maxVal)
```

This method is designed to take in an array of velocity vectors, extract each individual vector ❶, and pass it along to the `limitVec()` method, which looks like this:

```
def limitVec(self, vec, maxVal):

    """limit the magnitude of the 2D vector"""

    ❶ mag = norm(vec)

    if mag > maxVal:

        ❷ vec[0], vec[1] = vec[0]*maxVal/mag, vec[1]*maxVal/mag
```

You use the `norm()` function from the `numpy` library to calculate the magnitude of the vector ❶. If it exceeds the maximum, you scale the x and y portions of the vector in proportion to the vector's magnitude ❷. The maximum value was defined as `self.maxRuleVel = 0.03` as part of the initialization of the `Boids` class.

Influencing the Simulation

The core rules in the Boids simulation will cause the boids to automatically exhibit flocking behavior. But let's make things more interesting by allowing the user to influence the simulation as it runs. Specifically, you'll create the ability to add boids to the flock or make the flock scatter with the click of your mouse.

The first step to inject events into the simulation while it's running is to add an *event handler* to the `matplotlib` canvas. This is a piece of code that calls a function every time a certain event, such as a mouse click, happens. Here's how to do it:

```
cid = fig.canvas.mpl_connect('button_press_event', boids.buttonPress)
```

You use the `mpl_connect()` method to add a button press event handler to the `matplotlib` canvas. This handler will call the `buttonPress()` method of the `Boids` class every time a mouse button is pressed in the simulation window. Next, you need to define the `buttonPress()` method.

Adding a Boid

The first part of the `buttonPress()` method adds a boid to the simulation at the location of your cursor and assigns that boid a random velocity, when the *left* mouse button is pressed.

```
def buttonPress(self, event):

    """event handler for matplotlib button presses"""

    # left-click to add a boid

    ❶ if event.button is 1:

        ❷ self.pos = np.concatenate((self.pos,

                                   np.array([[event.xdata, event.ydata]]),

                                   axis=0)

        # generate a random velocity

        angles = 2*math.pi*np.random.rand(1)

        v = np.array(list(zip(np.sin(angles), np.cos(angles))))

        ❸ self.vel = np.concatenate((self.vel, v), axis=0)

        ❹ self.N += 1
```

First you ensure that the mouse event is a left-click ❶. Then you append the mouse location given by (`event.xdata`, `event.ydata`) to the array of boid positions ❷. You also generate a random velocity vector, add it to the array of boid velocities ❸, and increment the count of boids by 1 ❹.

Scattering the Boids

The three simulation rules keep the boids in a flock as they move around. But what happens when the flock is disturbed? To simulate this situation, you can introduce a “scatter” effect: when you right-click in the simulation window, the flock will scatter from the location of the click. You can think of this as how the flock might respond to the sudden appearance of a predator or a loud noise that spooks the birds. You implement this effect as a continuation of the `buttonPress()` method:

```
# right-click to scatter boids
```

```
❶ elif event.button is 3:
```

```
    # add scattering velocity
```

```
    self.vel += 0.1*(self.pos - np.array([[event.xdata, event.ydata]]))
```

Here you check whether the mouse button press is a right-click event ❶. If so, you change the velocity for every boid by adding a vector that points away from the point where the disturbance arose (that is, where the mouse was clicked). You calculate this vector much like you calculated the displacement vectors for the separation rule. If P_i is the position of a boid and P_m is the point where the mouse was clicked, $P_i - P_m$ is a vector that points away from the mouse click. You multiply this vector by `0.1` to keep the magnitude of the disturbance small. Initially, the boids will fly away from that point, but as you’ll see, the three rules prevail, and the boids will coalesce again as a flock.

Incrementing the Simulation

At each time step in the simulation, you need to apply the rules to calculate the boids' new velocities, update the boids' positions based on those velocities, enforce the boundary conditions, and redraw everything in the display window. You can coordinate all this activity from the `tick()` function, which will be called at each frame of the `matplotlib` animation.

```
def tick(frameNum, pts, head, boids):
```

```
    """update function for animation"""
```

```
    boids.tick(frameNum, pts, head)
```

```
    return pts, head
```

The stand-alone `tick()` function simply calls the `tick()` method of the `Boids` class. The latter is defined as follows:

```
def tick(self, frameNum, pts, head):
```

```
    """update the simulation by one time step"""
```

```
    # apply rules
```

```
    ❶ self.vel += self.applyRules()
```

```
    ❷ self.limit(self.vel, self.maxVel)
```

```
    ❸ self.pos += self.vel
```

```
    ❹ self.applyBC()
```

```
    # update data
```

```
    ❺ pts.set_data(self.pos.reshape(2*self.N)[:2],
```

```
self.pos.reshape(2*self.N)[1::2])
```

```
⑥ vec = self.pos + 10*self.vel/self.maxVel
```

```
⑦ head.set_data(vec.reshape(2*self.N)[::2],
```

```
vec.reshape(2*self.N)[1::2])
```

This method is where everything comes together. You apply the boid rules using the `applyRules()` method that we already looked at ❶. Then you limit the computed velocities of the boids using the `self.maxVel` threshold ❷. (Even though you limited the velocity vector generated by each individual rule, the overall velocity determined by adding all three rules together may still be too large.) You next compute the updated positions of the boids by adding the new velocity vectors to the old array of positions ❸. For example, if a boid was at position `[0, 0]` and has a velocity vector of `[1, 1]`, its new position after one time step would be `[1, 1]`. You apply the boundary conditions for the simulation by calling `applyBC()` ❹.

The call to `pts.set_data()` ❺ updates the `matplotlib` axis with the boids' new positions. The `[::2]` picks out the even-numbered elements (x-axis values) from the `pos` array, and the `[1::2]` picks out the odd-numbered elements (y-axis values). This will redraw the larger circles representing the boids' bodies. Next, you need to draw the smaller circles representing the boids' heads. You calculate the position of each boid's head so it will point in the boid's direction of motion by applying the $H = P + k \times V$ formula discussed earlier ❻. Recall that P is the center of a boid's body, k is a constant representing the distance from the center of the body to the center of the head (you use a value of 10 units), and V is the boid's velocity. Once you have the new head positions, you draw them via the same technique you used to draw the bodies ❼.

Parsing Arguments and Instantiating the Boids

The program's `main()` function begins by handling command line arguments and instantiating the `Boids` class:

```
def main():

    # use sys.argv if needed

    print('starting boids...')

    parser = argparse.ArgumentParser(description="Implementing
    Craig

                                Reynolds's Boids...")

    # add arguments

    ❶ parser.add_argument('--num-boids', dest='N', required=False)

    args = parser.parse_args()

    # set the initial number of boids

    ❷ N = 100

    if args.N:

        N = int(args.N)

    # create boids

    ❸ boids = Boids(N)
```

You use the familiar `argparse` module to create a command line option for setting the initial number of boids in the simulation ❶. If no argument is provided at the command line, the simulation defaults to 100 boids ❷. You set the simulation in motion by creating an object of the `Boids` class ❸.

The `main()` function continues with the code to create and animate a `matplotlib` plot. We've already discussed this code in [“Plotting the Boid's Body and Head”](#) on [page 84](#).

Running the Boids Simulation

Let's see what happens when you run the simulation. Enter the following:

```
$ python boids.py
```

The Boids simulation should start with all the boids clustered around the center of the window. Let the simulation run for a while, and the boids should start to flock as they form a pattern similar to the one shown in [Figure 5-4](#).

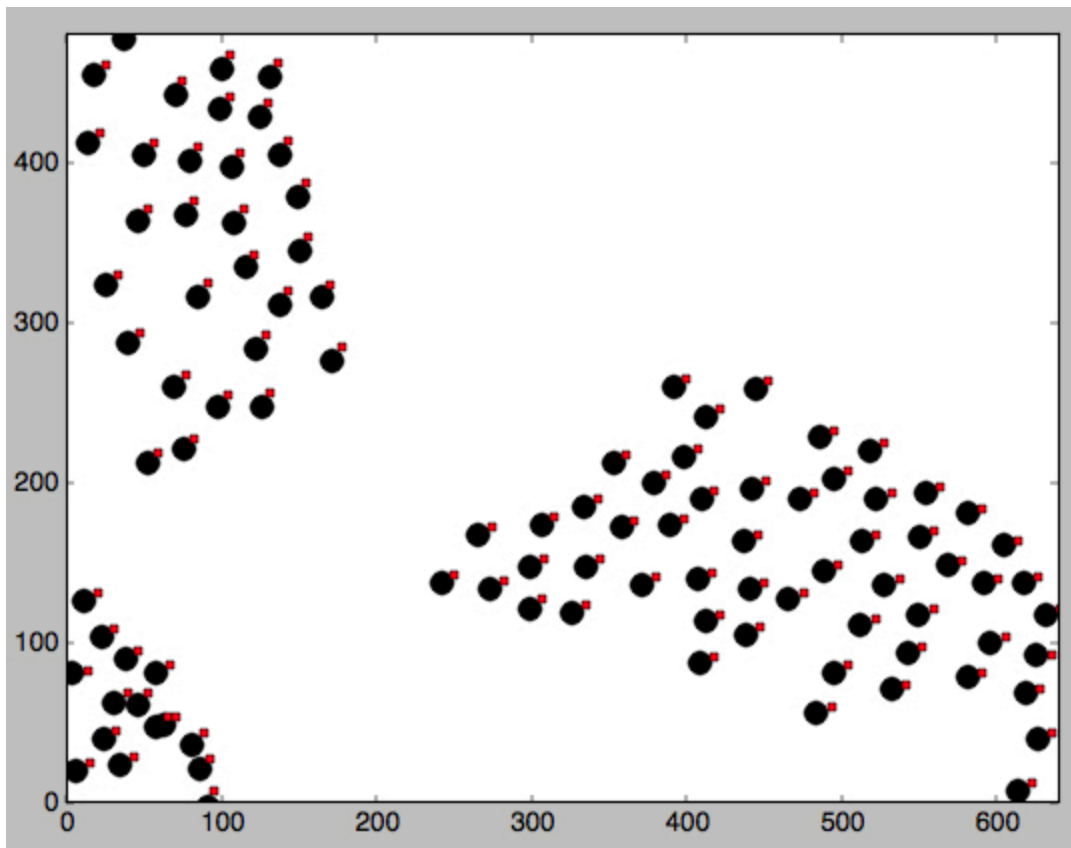


Figure 5-4: A sample run of the Boids simulation

Left-click the simulation window. A new boid should appear at that location, and its velocity should change as it encounters the flock. Now right-click. The flock should initially scatter from your cursor but then recoallesce.

Summary

In this project, you simulated the flocking of birds (or boids) using the three rules proposed by Craig Reynolds. You looked at how operating on an entire `numpy` array at once is much faster than performing the same operations inside an explicit loop. You used the `scipy.spatial` module to perform fast and convenient distance calculations, and you implemented a `matplotlib` trick that uses two markers to represent both the position and the direction of points. Finally, you added interactivity to the simulation in the form of an event handler to respond to button presses within the `matplotlib` plot.

Experiments!

Here are some ways you might further explore flocking behavior:

1. Implement obstacle avoidance for your flock of boids by writing a new method called `avoidObstacle()` and applying it right after you apply the three rules, as follows:

```
self.vel += self.applyRules()  
self.vel += self.avoidObstacle()
```

The `avoidObstacle()` method should use a predefined tuple (x, y, R) to add an additional velocity term to a boid, pushing it away from the obstacle location (x, y) , but only when the boid is within radius R of the obstacle. Think of this as the distance at which a boid sees the obstacle and steers away from it. You can specify the (x, y, R) tuple using a command line option.

2. What happens when the boids fly through a strong gust of wind? Simulate this by adding a global velocity component to all the boids

at random time steps in the simulation. The boids should temporarily be affected by the wind but return to the flock once the wind stops.

The Complete Code

Here's the complete code for the Boids simulation:

```
"""
```

```
boids.py
```

```
An implementation of Craig Reynolds's Boids simulation.
```

```
Author: Mahesh Venkitachalam
```

```
"""
```

```
import argparse
```

```
import math
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import matplotlib.animation as animation
```

```
from scipy.spatial.distance import squareform, pdist
```

```
from numpy.linalg import norm
```

```
width, height = 640, 480
```

```
class Boids:
```

```
"""class that represents Boids simulation"""

def __init__(self, N):

    """initialize the Boids simulation"""

    # init position & velocities

    self.pos = [width/2.0, height/2.0] +

        10*np.random.rand(2*N).reshape(N, 2)

    # normalized random velocities

    angles = 2*math.pi*np.random.rand(N)

    self.vel = np.array(list(zip(np.cos(angles), np.sin(angles))))

    self.N = N

    # min dist of approach

    self.minDist = 25.0

    # max magnitude of velocities calculated by "rules"

    self.maxRuleVel = 0.03

    # max magnitude of final velocity

    self.maxVel = 2.0

def tick(self, frameNum, pts, head):

    """update the simulation by one time step"""

    # apply rules
```

```
self.vel += self.applyRules()

self.limit(self.vel, self.maxVel)

self.pos += self.vel

self.applyBC()

# update data

pts.set_data(self.pos.reshape(2*self.N)[:2],

              self.pos.reshape(2*self.N)[1::2])

vec = self.pos + 10*self.vel/self.maxVel

head.set_data(vec.reshape(2*self.N)[:2],

              vec.reshape(2*self.N)[1::2])

def limitVec(self, vec, maxVal):

    """limit magnitude of 2D vector"""

    mag = norm(vec)

    if mag > maxVal:

        vec[0], vec[1] = vec[0]*maxVal/mag, vec[1]*maxVal/mag

def limit(self, X, maxVal):

    """limit magnitude of 2D vectors in array X to maxVal"""

    for vec in X:

        self.limitVec(vec, maxVal)
```



```
def applyBC(self):

    """apply boundary conditions"""

    deltaR = 2.0

    for coord in self.pos:

        if coord[0] > width + deltaR:

            coord[0] = - deltaR

        if coord[0] < - deltaR:

            coord[0] = width + deltaR

        if coord[1] > height + deltaR:

            coord[1] = - deltaR

        if coord[1] < - deltaR:

            coord[1] = height + deltaR

    def applyRules(self):

        # get pairwise distances

        self.distMatrix = squareform(pdist(self.pos))

        # apply rule #1 - separation

        D = self.distMatrix < self.minDist

        vel = self.pos*D.sum(axis=1).reshape(self.N, 1) - D.dot(self.pos)

        self.limit(vel, self.maxRuleVel)
```

```
# different distance threshold

D = self.distMatrix < 50.0

# apply rule #2 - alignment

vel2 = D.dot(self.vel)

self.limit(vel2, self.maxRuleVel)

vel += vel2;

# apply rule #1 - cohesion

vel3 = D.dot(self.pos) - self.pos

self.limit(vel3, self.maxRuleVel)

vel += vel3

return vel

def buttonPress(self, event):

    """event handler for matplotlib button presses"""

    # left-click - add a boid

    if event.button == 1:

        self.pos = np.concatenate((self.pos,

                                    np.array([[event.xdata, event.ydata]])),

                                    axis=0)

        # random velocity
```

```
angles = 2*math.pi*np.random.rand(1)

v = np.array(list(zip(np.sin(angles), np.cos(angles))))

self.vel = np.concatenate((self.vel, v), axis=0)

self.N += 1

# right-click - scatter

elif event.button == 3:

    # add scattering velocity

    self.vel += 0.1*(self.pos - np.array([[event.xdata, event.ydata]]))

def tick(frameNum, pts, head, boids):

    """update function for animation"""

    boids.tick(frameNum, pts, head)

    return pts, head

# main() function

def main():

    # use sys.argv if needed

    print('starting boids...')

    parser = argparse.ArgumentParser(description=

        "Implementing Craig Reynolds's Boids...")

    # add arguments
```

```
parser.add_argument('--num-boids', dest='N', required=False)

args = parser.parse_args()

# number of boids

N = 100

if args.N:

    N = int(args.N)

# create boids

boids = Boids(N)

# setup plot

fig = plt.figure()

ax = plt.axes(xlim=(0, width), ylim=(0, height))

pts = ax.plot([], [], markersize=10,

               c='k', marker='o', ls='None')

head, = ax.plot([], [], markersize=4,

                 c='r', marker='o', ls='None')

anim = animation.FuncAnimation(fig, tick, fargs=(pts[0], head,
boids),

                               interval=50)

# add a "button press" event handler
```

```
cid = fig.canvas.mpl_connect('button_press_event',  
boids.buttonPress)
```

```
plt.show()
```

```
# call main
```

```
if __name__ == '__main__':
```

```
    main()
```
